


Lec 23

29 Octo, 2024

2nd Octo
2023

Execution Content :-

How actually your code is being run?

Please refer Shreyan's sir class notes $\Rightarrow$ (Lec 14 6 July 2024)

Page no : 36

$\rightarrow$ For Execution Content

Note: Iss notebook mei jis tarika se sir ne GEC banana dikhaya hai, waise hi interview mei batana hai.

Q what is the minimum line of code you can write in JS?

Ans empty js file is the shortest program of js.

jab bhi js code run hoga $\rightarrow$ global execution context create hoga and inside GEC, we have 2 different phases

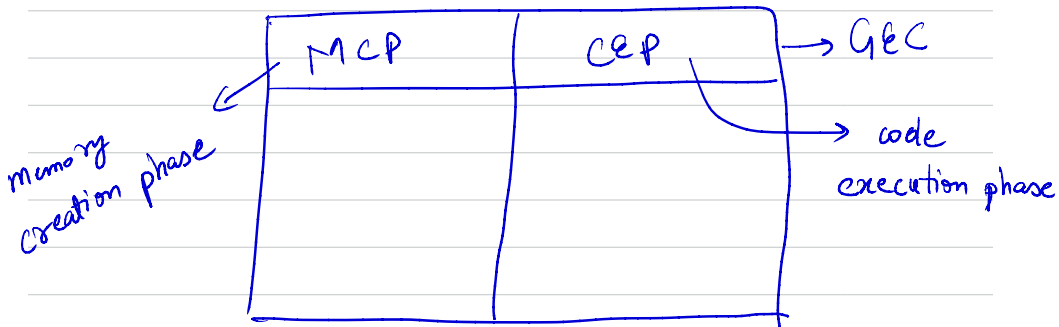
- $\rightarrow$ ① memory creation phase
- $\rightarrow$ ② code execution phase

① memory creation phase $\rightarrow$ memory de raha hoga

Inside the memory creation phase, all the variables

and functions → they all get memory separately.

② code execution phase is the phase where the code runs line by line.



MCP phase mei not a single line of code is executed. Iss phase mei we skim out functions & variables.

CEP phase mei line by line code execute hoga.

③

```
var userName = 'samarth';  
  
var person = {  
  name: 'hi',  
  lastName: 'there'  
};  
  
console.log(userName);
```

↪ iss code ka GEC banana.
& objects

MCP mei sabhi variables will be assigned undefined kyunki code execution shuru nahi hua. But function will have its definition with him in MCP phase.

GEC

MCP	CEP
userName: undefined person: undefined	



MCP	CEP
userName: Samarth person: { ... }	Clg (userName)

CEP phase mei code line by line execute hoga.
and variable & object ko values assign
ho jayegi in memory. Then console mei samarth
print / out put aa jayega.

console

Samarth

Once the entire code execution is done completely,
The global execution content will look like this
and it will be destroyed also.

MCP	CEP
userName: Samarth person: { ... }	

→ GEC



Ab GEC destroy ho jayega.

Q

```
var userName = 'samarth';  
  
function sum(){  
  var num1 = 10;  
  var num2 = 14;  
  
  return num1+num2;  
}  
  
sum();  
  
console.log(userName);
```

Interview mei iss
tarika se samjhana
hai

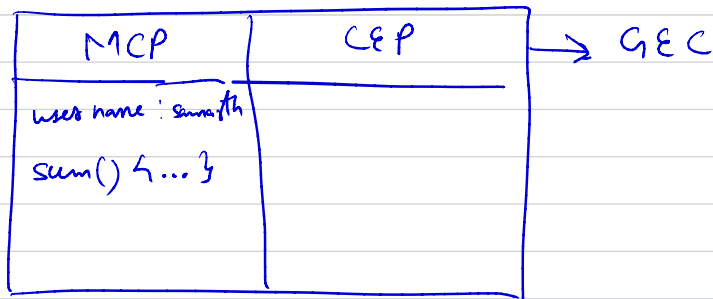
① jab bhi js code run hoga → global execution context create hoga and inside GEC, we have 2 different phases

- ① memory creation phase
- ② code execution phase

② Sabhi variables, functions, objects ko skim out / bahar nikal liya jayega. function will have its definition but other variables will be assigned undefined. Undefined isliye kyunki code execution shuru nahi hua.

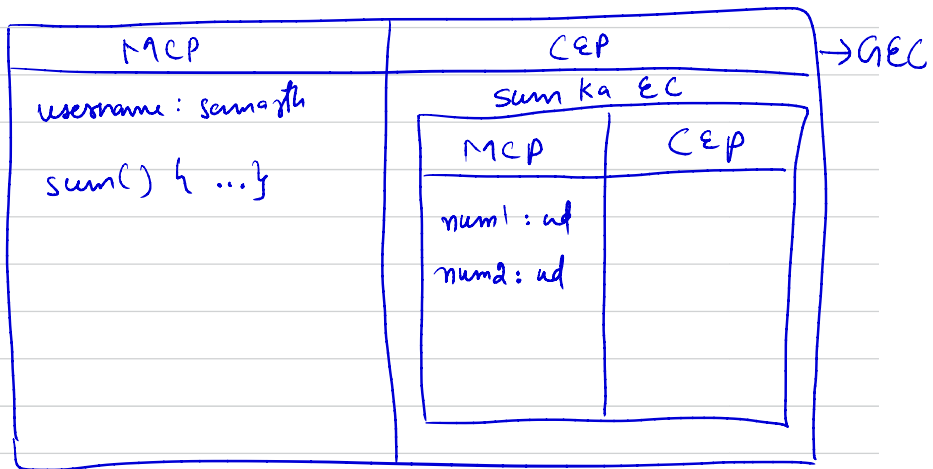
MCP	CEP	→ GEC
user name : ud sum : 4...}		

③ code execution begins



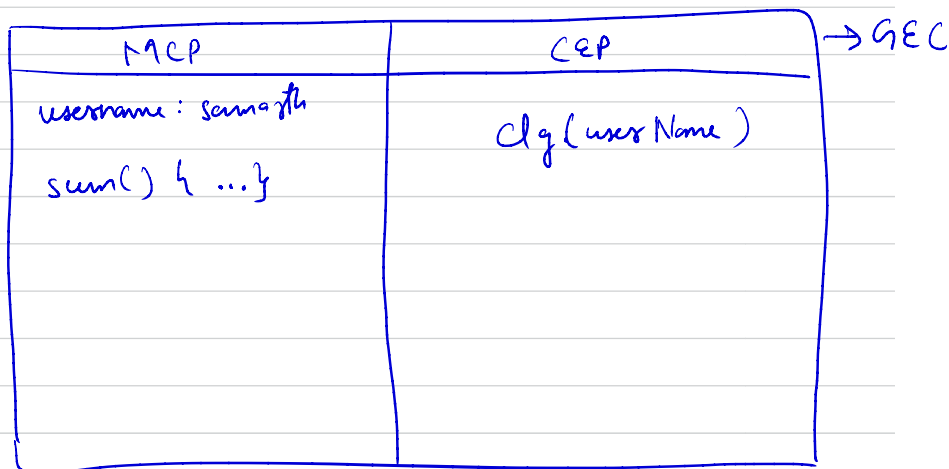
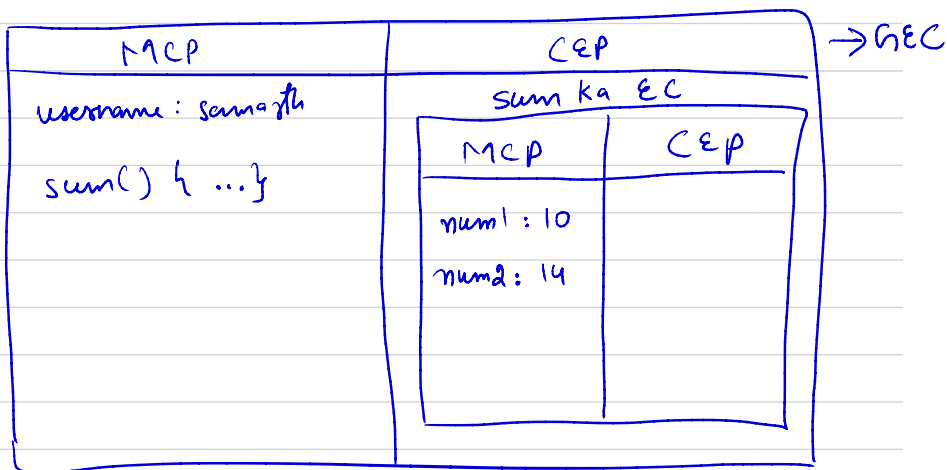
username will be assigned "samarth" in memory.

Ab sum() call hoga. Ab naya execution context banega. (This is not global execution context)



Then MCP phase mei num1 & num2 ko undefined milega.

Then CEP phase chalega num1 & num2 will be assigned values. Then num1 + num2 will be returned. Taise hi returned value huyi, sum function ka execution content khatam or destroy ho jayega.

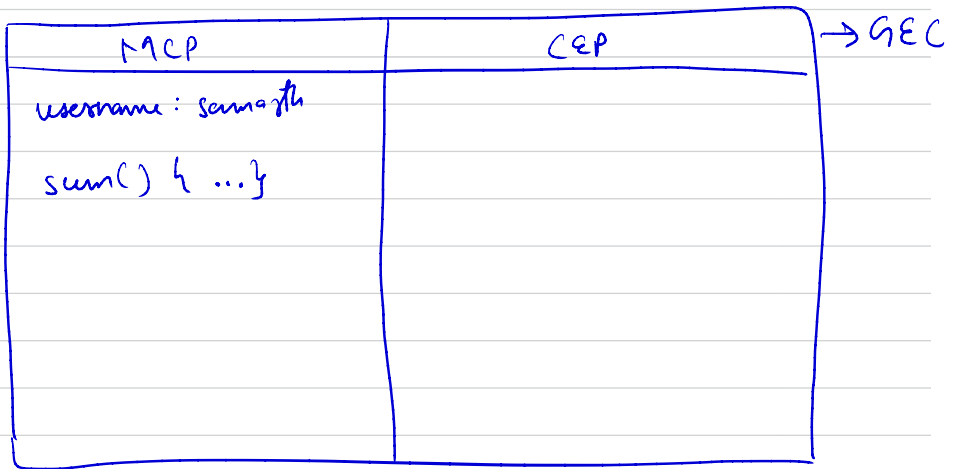


Ab console ho jayega "samarth"



console

samarth



Once the entire code execution is done completely,
The global execution content will look like this
and it will be destroyed also.

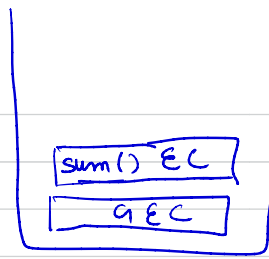
Q

```

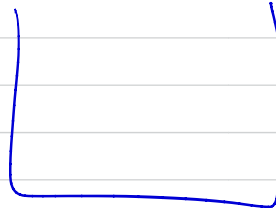
var userName = 'samarth'; l1
function sum(){
  var num1 = 10; l2
  var num2 = 14; l3
  return num1+num2; l4
}
sum(); l5
console.log(userName); l6
  
```

→ Hamne upar iss code ka
GEC banaya. Ab same code
ka Call Stack banao.





empty
call stack ⇐



Explanation from gpt :-

Call Stack Steps

Here's how the call stack changes step-by-step:

Step 1: Global Execution Context (GEC) Created

- Call Stack: [GEC]

Step 2: Memory and Execution Phases

- `userName` is assigned `'samarth'`.

Step 3: Calling `sum()`

- A new Execution Context for `sum` is created and pushed onto the stack.
- Call Stack: [GEC, `sum()` Execution Context]

Inside `sum()` :

1. `var num1 = 10;` assigns `10` to `num1`.
2. `var num2 = 14;` assigns `14` to `num2`.
3. `return num1 + num2;` calculates `24` and returns it to the global context.

• After `sum()` completes:

- `sum()` Execution Context is **popped** from the stack.
- Call Stack: `[ GEC ]`

Step 4: `console.log(userName);`

- Logs `'samarth'`.
- End of Code: Global Execution Context is popped from the stack.
- Final Call Stack: `[]`

Most Asked Questions :-

①

```
var a = 20;

function meraFunction(){
  console.log("inside
  meraFunction")
}

console.log(a);
meraFunction();
```

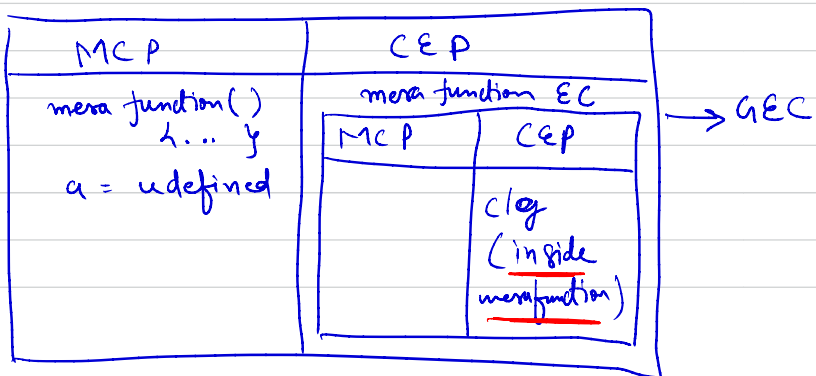
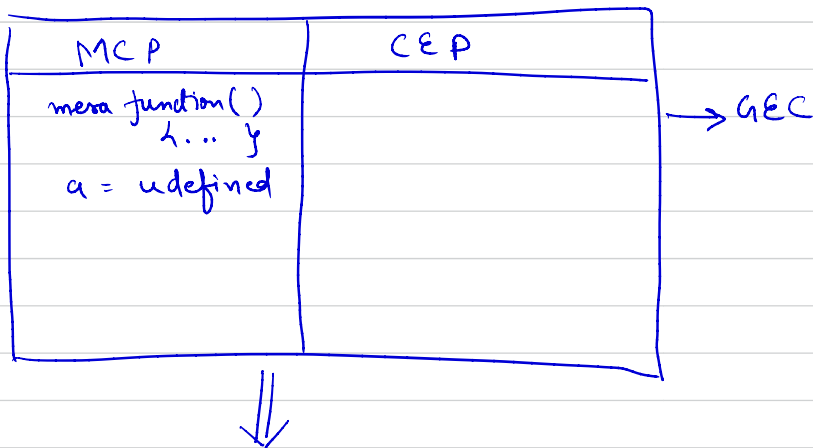
o/p $\Rightarrow$ 20
inside mera Function

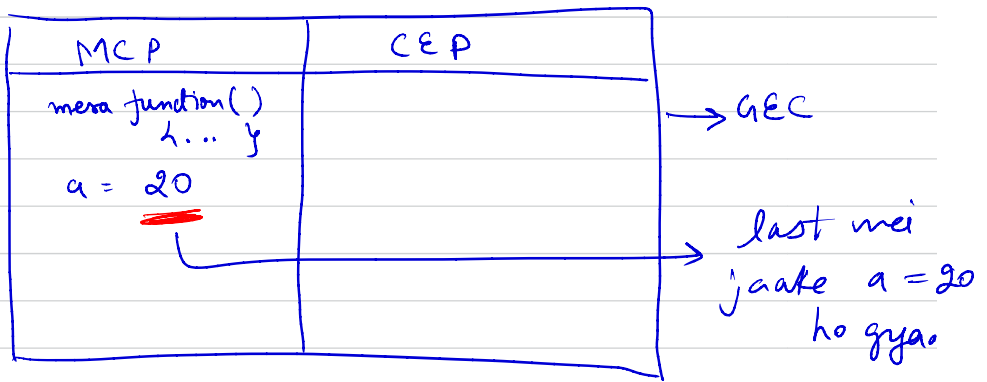
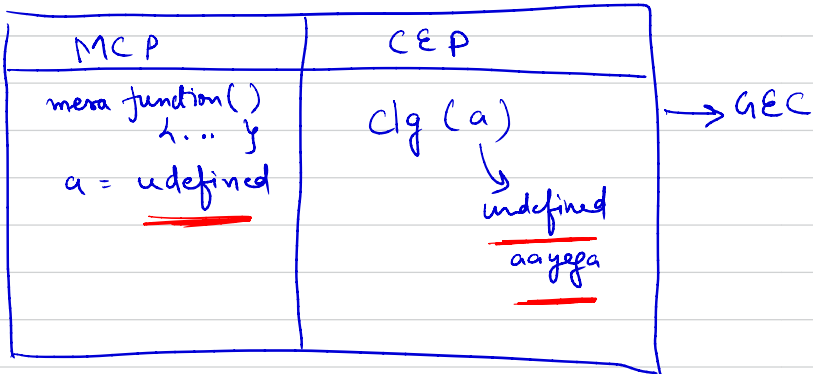
2

```
meraFunction(); //use  
console.log(a); //use  
  
var a = 20; //define  
  
function meraFunction(){  
  console.log("inside  
  meraFunction")  
}
```

O/p → inside mera function
undefined

G EC banao iska :-





Once the entire code execution is done completely,
The global execution content will look like this
and it will be destroyed also.

This was hoisting →

→ incomplete definition

// whenever we access the variable or the function before its declaration then it is called hoisting

{ Please refer Lec 14 6 July 2024 notes

Hoisting is a JavaScript mechanism where variable and function declarations are moved to the top of their containing scope during the compilation phase, before the code is executed. This means that you can refer to variables and functions before you actually declare them in the code.

```
l1 meraFunction(); //use  
l2 console.log(a); //use  
  
var a = 20; //define  
  
function meraFunction(){  
    console.log("inside meraFunction")  
}
```

Hoisting ke karan hi l1 and l2 par error nahi aaya. Kyunki mera function function ki definition is already there in MEP and a is undefined in MEP.

In JS, var declaration is hoisted with a default initialization of undefined.

let and const are hoisted too but without any initialization.

var Declarations:

Hoisting: Yes, var declarations are hoisted.

Initialization: During hoisting, var variables are initialized with undefined.

let and const Declarations:

Hoisting: Yes, let and const declarations are also hoisted.

Initialization: let and const variables are not initialized during hoisting. They remain uninitialized and are in the Temporal Dead Zone (TDZ) until the execution reaches the line where they are declared.

